
Software Requirements Specification for Cow-puter Vision-Animal Detection on Dairy Farms

Version 0

Team 21

COMPSCI 4ZP6

Dr.Mehdi Moradi

October 12, 2025

Software Requirements Specification for Cow-puter Vision-Animal Detection on Dairy Farms.....	1
1. SRS.....	1
1.1. Team Members.....	1
1.2. Document Information.....	1
1.2.1 Contributions.....	1
1.2.2 Revision History.....	1
1.2.3 List of Abbreviations and Notations.....	2
1.3. Purpose of the Project.....	2
1.3.1 Background.....	2
1.3.2 Problem Description.....	2
1.3.3 Goal of Project.....	2
1.4. Client and Stakeholders.....	3
1.5. Project Constraints.....	3
1.5.1 Functional constraint.....	3
1.5.2 Internet constraint.....	3
1.5.3 Deployment constraint.....	3
1.5.4 Hardware constraint.....	3
1.6. Functional Requirements.....	4
1.6.1 Priority 0 (Minimum Viable Product).....	4
1.6.1.1 ML Model.....	4
1.6.1.2 Backend API & Core Logic.....	4
1.6.1.3 Frontend & User Interface.....	4
1.6.2 Priority 1.....	4
1.6.2.1 ML Model.....	4
1.6.2.2 Backend API & Core Logic.....	5
1.6.2.3 Frontend & User Interface.....	5
1.6.3 Priority 2.....	5
1.6.3.1 ML Model.....	5
1.6.3.2 Backend API & Core Logic.....	5
1.6.3.3 Frontend & User Interface.....	6
1.6.4 Priority 3.....	6
1.6.4.1 Backend API & Core Logic.....	6
1.6.4.2 Frontend & User Interface.....	6
1.7. Data and Metrics.....	6
1.7.1 Data Sources and Links.....	6
1.7.2 Dataset Planning and Balancing.....	7
1.7.3 Dataset Splits.....	7
1.7.4 Evaluation Plan.....	8

1.7.5 Expected Targets.....	8
1.8. Non-functional Requirements.....	9
1.8.1 Look and Feel Requirements.....	9
1.8.2 Usability and Humanity Requirements.....	9
1.8.3 Performance and Speed Requirements.....	9
1.8.4 Maintainability and Support Requirements.....	9
1.8.5 Security and Privacy Requirements.....	9
1.8.6 Compliance Requirements.....	9
1.8.7 Legal Requirements.....	9
1.9. Risks and Issues Predicted.....	10
1.9.1 Hardware.....	10
1.9.2 Model.....	10
1.9.3 Management.....	10
2. Project Development Plan.....	11
2.1 Team Meeting and Communication Plan.....	11
2.2 Team Member Roles.....	11
2.3 Workflow.....	12
2.4 Proof of Concept Demonstration.....	12
2.4.1 Backend.....	12
2.4.1 Frontend.....	12
2.5 Technology.....	13
2.5.1 Frontend.....	13
2.5.2 Backend.....	13
2.5.3 Database.....	14
2.5.4 ML Libraries.....	14
2.5.5 Infrastructure.....	14
2.5.6 Others.....	14
2.6 Project Scheduling.....	15

1. SRS

1.1. Team Members

Group Members	Student Number	Contact	Roles
Xianpi Duan	400381987	duanx14@mcmaster.ca	Backend/Frontend Developer, Code Lead
Xiaozhao Duan	400381647	duanx15@mcmaster.ca	Frontend/Backend Developer
Zicheng Li	400359321	li1276@mcmaster.ca	Program Manager Developer
Bozhi Zhang	400327039	zhanb50@mcmaster.ca	Machine Learning Lead
Zhanxu Ye	400390561	yez69@mcmaster.ca	Developer

1.2. Document Information

1.2.1 Contributions

Group Members	Contributions (Sections)
Xianpi Duan	1.2, 2.1, 2.4, 2.5
Xiaozhao Duan	1.6, 1.8, 2.6
Zicheng Li	1.5, 2.2, 2.3
Bozhi Zhang	1.2, 1.6, 1.7
Zhanxu Ye	1.1, 1.2, 1.3, 1.4, 1.9

1.2.2 Revision History

Version	Authors	Description	Date
0	All	Created first draft of document.	October 12, 2025

1.2.3 List of Abbreviations and Notations

AI	Artificial Intelligence	FPR	False Positive Rate
ML	Machine Learning	ROC	Receiver Operating Characteristic curve
YOLO	You Only Look Once (object detection model)	AUC	Area Under the Receiver Operating Characteristic Curve
ID	Identification Number	CBVD-5	Cow Behaviour Video Dataset
API	Application Programming Interface	CVB	A Video Dataset of Cattle Visual Behaviours
CRUD	Create, Read, Update, Delete	RTSP	Real-Time Streaming Protocol
UI	User interface	FPS	Frames Per Second
TPR	True Positive Rate	POC	Proof of Concept

1.3. Purpose of the Project

1.3.1 Background

- Dairy farming is an important part of the agricultural industry, but the health monitoring of dairy cows still mainly relies on manual observation, which is both time-consuming and prone to errors. As the size of the cattle herd expands, manual monitoring becomes inefficient and costly.
- Sensor-based systems (such as CowScout) can monitor activities, but they are expensive and require frequent maintenance. With the development of computer vision and artificial intelligence technologies, it is now possible to directly analyze the behaviour of dairy cows through video, achieving automated health monitoring.

1.3.2 Problem Description

- Manual identification is inefficient and prone to errors, while ear tags or sensors are costly and may cause discomfort to cows. Farmers need a faster, contactless and scalable way to identify individuals.

1.3.3 Goal of Project

- The objective of this project is to design and construct a web-based artificial intelligence dairy cow health monitoring system. The system will utilize the

YOLO model to automatically detect and classify the behaviors of dairy cows, such as standing, lying down, eating and walking, thereby helping to identify potential health issues and supporting the ranch in making management decisions at an early stage.

The final deliverables of the project include a trained YOLO model, a front-end dashboard for data visualization and result management, and a back-end server responsible for model inference, data processing and storage. The back end also provides API interfaces to support web-based interaction and future data expansion. The web application will allow users to subscribe to streams, browse detection results, and generate behavior analysis reports.

This project not only aims to enhance the efficiency and accuracy of dairy cow health monitoring, but also is committed to providing a scalable and user-friendly digital platform for farm management. By integrating machine learning, web development and agricultural applications, this system will promote the construction of smart ranches and lay a technical foundation for future precise livestock management solutions.

1.4. Client and Stakeholders

- Client: Dr. Mehdi Moradi; Dr. Amirhossein Sabour; Luke(Project Supervisor); CATTLEytics
- Stakeholders: COMPSCI 4ZP6 Team 21; Dairy farm operators; Dr. Mehdi Moradi; Dr. Amirhossein Sabour; CATTLEytics

1.5. Project Constraints

1.5.1 Functional constraint

- This application will combine a website-based frontend and an AI model as a backend. The user can use the website to monitor the camera video from the farm in real-time, along with AI model predictions.

1.5.2 Internet constraint

- The Internet at the farm is slow. It constrains us to at most 5 Mbps, which also limits our application deployment and hardware.

1.5.3 Deployment constraint

- The client needs to deploy this app on-premises since the Internet speed is constrained.
- The application needs to be deployed on the machine located in the farm.

1.5.4 Hardware constraint

- The machine needs to have at least 16 GB of memory, with a 200 GFLOPs GPU.

- It needs to have a modern 6-core 12-thread CPU.

1.6. Functional Requirements

1.6.1 Priority 0 (Minimum Viable Product)

1.6.1.1 ML Model

- The model is able to detect cows in a video frame
- The model is able to classify the following behaviour of a detected cow:
 - Standing
 - Lying down
 - Feeding
 - Drinking
- The model is able to track detected cows across consecutive video frames to maintain a unique identity for the duration of a video clip

➤ Datasets:

- CBVD-5
- CVB
- Beef Cattle Behaviour Data Set

1.6.1.2 Backend API & Core Logic

- The backend must be able to receive a video stream from a camera
- The backend must be able to use the trained model to infer on the received frames
- The backend must be able to provide a video stream with the model inference outputs to the frontend
- The system must store the received video frames in the local file system
- The database must store the video frames' path and the corresponding inference outputs

1.6.1.3 Frontend & User Interface

- The system must provide a web-based frontend that requires user authentication
- The frontend must be able to play from a video stream
- The frontend must be able to overlay bounding boxes and labels on the playing video
- The frontend must provide a dashboard showing all detected cows' current behaviours and their lasting time

1.6.2 Priority 1

1.6.2.1 ML Model

- The model will be fine-tuned on a custom dataset of farm-specific video footage to improve accuracy

- Datasets:
 - Live camera footage provided by the supervisor

1.6.2.2 Backend API & Core Logic

- The backend must provide APIs for the frontend to retrieve behaviour data with filtering conditions
- The backend must provide APIs for the CRUD operations of alerts
- The backend must be able to send proactive notifications (e.g. Email) for alerts
- The backend must store triggered alerts' triggering conditions and relevant data

1.6.2.3 Frontend & User Interface

- Users can filter overlaid bounding boxes by selecting labels
- The dashboard must display detailed behaviour data of each detected cow
- The dashboard must display various visualizations/charts of behaviour data for all cows
- The dashboard must display various visualizations/charts of behaviour data for a single cow
- The frontend must be able to allow the user to set up alerts for custom conditions (E.g. lying down for over 5 hours) with corresponding triggering actions (e.g. Email, Logging)
- The frontend must display configured alerts, sorted by priority and timestamp
- The frontend must display triggered alerts, with triggering conditions and relevant data

1.6.3 Priority 2

1.6.3.1 ML Model

- The system is able to classify additional behaviours:
 - Ruminating
 - Heat (mounting)
- The system incorporates a re-identification model to maintain cow identity across different cameras or after long-term occlusion

1.6.3.2 Backend API & Core Logic

- The backend should be able to provide a video stream with the stored inference outputs and frames
- The backend should be able to map the bounding boxes' coordinates to a 2D coordinate system
- The backend should be able to aggregate the needed data for reports
- The backend should be able to generate a video file with the stored inference outputs and frames

1.6.3.3 Frontend & User Interface

- The frontend should have a “Reporting” section with customizable date ranges and data
- The frontend should have a “Playback” section
- Users can export and download an overlaid video within the “Playback” section with custom date ranges and labels
- The frontend could feature an interactive 2D floor plan of the barn, showing real-time cow locations and behaviours
- The floor plan should have a “time machine” feature to show historical cow locations and behaviours
- Users can click on the boxes bounded cow from the real-time video feed or floor plan to open a detailed cow profile

1.6.4 Priority 3

1.6.4.1 Backend API & Core Logic

- The backend could generate PDF files from reports

1.6.4.2 Frontend & User Interface:

- The frontend could display and download PDF files
- The frontend could feature an interactive 3D floor plan of the barn, showing real-time cow locations and behaviours
- Localization for French

1.7. Data and Metrics

1.7.1 Data Sources and Links

The primary training data will be behaviour-labelled video clips. We will use several public cattle behaviour datasets to meet those needs:

- CBVD-5 (Cow Behaviour Video Dataset):
Behaviours: standing, lying down, feeding, drinking, rumination
Dataset: <https://www.kaggle.com/datasets/fandaoerji/cbvd-5cow-behavior-video-dataset>
Paper: <https://www.nature.com/articles/s41598-024-65953-x>
Captured with seven cameras, the dataset contains 687 video clips and 206,100 image frames under varying illumination and day–night conditions. CBVD-5 provides per-frame behaviour labels for cows and can also be used for tasks such as object detection. We will primarily use its behaviour labels to train the model’s core recognition capability.

- CSIRO Data Access Portal – CVB: A Video Dataset of Cattle Visual Behaviours: Behaviours: grazing, walking, running, ruminating-standing, ruminating-lying, resting-standing, resting-lying, drinking, grooming, other Dataset:

<https://data.csiro.au/collection/csiro%3A58916v1>

Paper:

<https://arxiv.org/abs/2305.16555>

Recorded with four HD cameras, the dataset comprises 502 video clips, each 15 seconds (~450 frames). Experts annotated every cow in every frame with behaviour labels and assigned a unique ID within each clip. This dataset is valuable for validating recognition performance in complex scenes and for evaluating and tuning multi-object tracking algorithms.

- Beef Cattle Behaviour Data Set: Behaviours: standing, lying, drinking, feeding, ruminating Dataset:

<https://www.kaggle.com/datasets/lucyfirst/beef-cattle-behavior-data-set>

Paper:

<https://www.nature.com/articles/s41598-025-01948-6>

Collected at a cattle farm in Yunnan, China, the data were recorded for six beef cattle continuously over 168 hours, covering day and night, using a single fixed camera. A semi-automated annotation pipeline was used: YOLOv8 detected cows and ByteTrack associated coordinates and temporary IDs across frames; single-cow clips were then extracted and manually labelled with behaviours. The final dataset includes ~500 short clips (about 14 hours of video) and provides per-clip coordinates, identities, and behaviour labels.

1.7.2 Dataset Planning and Balancing

- Given potential class imbalance across behaviours, and real-world damage from cameras. For example, cows drinking occurs far less frequently than standing or feeding or camera lens smudges. We will apply data augmentation and resampling during training to balance classes.
- Measures include: random rotations, translations, flips, and brightness adjustments on training samples.
- For behaviours with too few examples, we'll generate extra augmented samples or apply targeted oversampling to reduce class imbalance. Oversampling simply means making rare classes appear more often in the training data. For example, by duplicating or lightly augmenting their clips, the model gets more balanced practice. If drinking clips are oversampled, the model sees enough cases to improve recognition accuracy for that class.
- We will also reserve a fixed proportion of data for validation and testing to avoid overfitting induced by aggressive oversampling.

1.7.3 Dataset Splits

- We will partition the aggregated data into training, validation, and test sets.

- Our current plan is to mix sources for training, while ensuring each dataset contributes a guaranteed portion to the test split to evaluate generalization across environments.
- For example, we may use a 7:2:1 with train:validation: test split.
- Model training will only be on the training set, and the final test set will be completely held out until the end. This allows us to assess performance on previously unseen and out-of-distribution data.

1.7.4 Evaluation Plan

During training and testing, we will use multiple quantitative metrics to assess model performance comprehensively:

- **Accuracy:** The proportion of correct predictions. In a multi-class task, accuracy reflects overall correctness. For example, if the model correctly identifies 90% of cattle behaviours in the test set, the accuracy is 90%. While intuitive, accuracy may understate problems on minority classes when the data are imbalanced.
- **Precision:** For a given behaviour class, precision is the proportion of predicted positives that truly belong to that class. It measures how well the model avoids false positives. Higher precision means more reliable, less “over-enthusiastic” predictions. For instance, for drinking, high precision means the model rarely mislabels non-drinking states as drinking.
- **Recall:** For a given behaviour class, recall is the proportion of actual positives that the model correctly identifies. It measures the ability to capture true targets. Higher recall means fewer misses. For drinking, a high recall indicates that most true drinking events are detected. Precision and recall often trade off against each other.
- **F1-score:** A single number that balances the two main error types for each class. We’ll examine F1 for every class. If the drinking F1 is clearly lower than the standing F1, it means the model either raises too many false alarms for drinking or misses too many real drinking cases. That suggests we may need more data or more targeted features for drinking.
- **ROC-AUC:** The ROC curve shows how you trade off TPR against FPR. For multi-class evaluation, we’ll use a one-vs-rest setup: pick one class, like drinking as the positive class and treat all others as negative, then repeat for feeding, etc. If a class has a lower AUC, it means the model is more likely to confuse that class with others. So this is a current weak point.

1.7.5 Expected Targets

Accuracy: $\geq 80\%$

Precision: $\geq 80\%$

Recall: $\geq 80\%$

F1-score: ≥ 0.80

ROC-AUC: ≥ 0.80

1.8. Non-functional Requirements

1.8.1 Look and Feel Requirements

- The UI shall have a clean, modern, and professional aesthetic suitable for a commercial agricultural setting.
- The UI should be fully responsive and usable on a tablet or mobile phone

1.8.2 Usability and Humanity Requirements

- A user with basic computer literacy shall be able to learn the core functionalities of the system (viewing alerts, checking cow status) with less than 30 minutes of training
- The system shall be navigable and fully functional using only a mouse or touchscreen, minimizing the need for keyboard input.

1.8.3 Performance and Speed Requirements

- The video processing pipeline shall be capable of processing at least 10 frames per second (fps) per camera stream
- The web dashboard shall load fully in under 5 seconds on a standard farm broadband connection

1.8.4 Maintainability and Support Requirements

- The codebase shall be modular, separating the video ingestion, ML inference, backend API, and frontend components. This allows for independent updates
- The system shall maintain detailed logs of model inference results, system errors, and user logs. Logs shall be retained for at least 30 days

1.8.5 Security and Privacy Requirements

- All access to the web dashboard shall require user authentication with a username and a strong password
- All communication between the frontend and backend, and between backend services, shall be encrypted using TLS 1.2 or higher

1.8.6 Compliance Requirements

- The system should meet at least the level A requirement of the WCAG guidelines

1.8.7 Legal Requirements

- The system shall clearly assert within its terms of use that all farm-generated data (video and behavioural) remains the exclusive property of the farm

1.9. Risks and Issues Predicted

1.9.1 Hardware

- The real-time monitoring cameras provided by the farm may have problems such as insufficient resolution, unstable network connection or environmental interference from lighting and weather
- Some videos may be blurry, blocked or unevenly illuminated, resulting in a decrease in the model's recognition accuracy
- Can't sustain 10 FPS per stream end-to-end

1.9.2 Model

- Training data may be insufficient to meet model requirements
- Missed/incorrect detections (standing/lying/feeding/drinking), which cause frequent false alerts
- ID tracking is unstable, for example, A cow's identity flips across frames

1.9.3 Management

- Team time conflicts may lead to delays in project progress.

2. Project Development Plan

2.1 Team Meeting and Communication Plan

The team has a private Teams channel that serves as the main communication hub. It is used for daily discussions and progress updates. All team documents (including meeting notes, project wiki, etc.) are collaboratively managed in Google Docs, which allows us real-time access and simultaneous contribution. Jira is the project management tool, where we assign responsibilities, track progress, and monitor milestones.

The team holds weekly meetings every Friday via MS Teams at 3:00 PM to review completed work from the past week and plan upcoming tasks. In the event of special circumstances (such as deadlocks or uncertainties regarding projects), virtual meetings may be escalated to in-person meetings. In such cases, Thode/Mills Library will serve as the meeting location.

A Discord server is also created for quick and informal communication with our supervisor, Luke Schuurman. We will meet biweekly with our supervisor via MS Teams to discuss project development and receive feedback from him.

Additionally, we meet twice a month with Dr. Mehdi Moradi and the TA to report our progress and demonstrate project outcomes. These meetings are scheduled in advance and held on MS Teams.

2.2 Team Member Roles

Group Members	Roles	Responsibilities
Xianpi Duan	Backend/Frontend Developer, Code Lead	Be in charge of backend development, assisting in frontend development, ensuring a nice-looking frontend and robust backend. Be responsible for reviewing all code in the PR.
Xiaozhao Duan	Frontend/Backend Developer	Developing the frontend user interface to allow the user to monitor the camera video from the farm. Developing and testing a backend API for the AI model.
Zicheng Li	Program Manager Developer	Managing the group's tasks and hosting the stand-up meeting. Assisting with the model training and testing.
Bozhi Zhang	Machine Learning Lead	Developing and testing the AI model for

		the project.
Zhanxu Ye	Developer	Assisting in data collection and training the model.

2.3 Workflow

We will use GitHub for version control. The **main** branch serves as the **production branch**; it will only be merged once the project is complete and requires approval from all team members. A **staging** branch will be created as a near replica of the production environment. Each team member will have their own branch named after them. Members can push code to their individual branches with clear commit messages. However, when merging code into the staging branch, they must create a pull request (PR) and assign **Code Lead Xianpi Duan** as the reviewer. Once the Code Lead approves, the PR can be merged. Code commits from the Code Lead must be reviewed by the **Program Manager**.

Process flow:

1. Run `git pull staging` to ensure you have the latest version of the code.
2. Push your code to your own branch with a clear commit message describing the additions or modifications.
3. Create a PR to the staging branch with a description outlining what changed and why, and assign the Code Lead as the reviewer.
4. Once approved, merge and close the PR.
5. When the project is complete, the Program Manager will create a PR from staging to main, add all team members as reviewers, and merge once approved.

Our team will use Jira for project management following agile methods. The Program Manager will manage sprints and host weekly stand-up meetings to ensure the team stays on schedule. The Program Manager can assign tasks to team members, who will then move the task to “In Progress” in Jira once they begin work. When a task is completed, the team member will check in the code and move the task to “Done.”

If code needs to be merged into the staging branch, the team member must inform the Program Manager to create a merge task in Jira. Once the pull request (PR) is closed, the corresponding task should also be moved to “Done.”

We will use Google Drive to store all machine learning data, including the dataset. This setup makes it easy for developers to access and extract the data when needed.

For model training, we plan to use the school's GPU resources, specifically the NVIDIA H100 GPU, to accelerate AI model training.

To ensure we meet all project requirements, all code must have a peer review, and every team member is responsible for writing additional unit tests for the code they develop. The project must achieve at least 75% test coverage to pass the testing phase.

2.4 Proof of Concept Demonstration

2.4.1 Backend

- Loop a short barn video as an RTSP stream.
- Inference service subscribes to RTSP, runs behaviour detection + tracking, and emits the frame and result.

2.4.1 Frontend

- The web video player plays the stream, and the overlay layer draws boxes/labels from the backend.
- The right-panel “Received Information” displays real-time information from the backend.
 - FPS
 - Behaviours in frame
 - Number of IDs in each frame to see the ID consistency

By this POC, we can validate:

- End-to-end real-time pipeline: camera/RTSP → decode → behavior detection → tracking → overlaid playback in the browser
- Runtime performance at the edge: quick on-screen checks of inference latency and FPS in the browser
- Tracking stability: visual verification of ID consistency across frames in the overlay and side panel

2.5 Technology

2.5.1 Frontend

- TypeScript and React with Next.js will be used for the frontend development. The UI will be built using Tailwind CSS combined with Headless UI components such as Shadcn. Recharts will be utilized for dashboards, while Three.js is planned for future Phase 3 (P3) to support 3D visualization features. Figma will serve as the primary tool for UI/UX design and prototyping.

2.5.2 Backend

Python will be used as the primary language for the backend. It will be separated into 3 parts. App side, Inference side and model side.

- **App:** Django will serve as the core application framework, handling authentication and providing data for the frontend. Django REST Framework (DRF) will be used to expose RESTful APIs, called by both the frontend and internal services.
- **Inference:** FastAPI will manage real-time video inference via RTSP streams. FFmpeg alongside OpenCV for frame decoding. After getting output from models, WebSocket/WebRTC channels will be used for live frame and metadata streaming. This service will also communicate directly with PostgreSQL using SQLAlchemy for data access.
- **Model:** We will train the Ultralytic YOLO/Detectron2 for cattle and their behaviour detection. DeepSORT/ByteTrack will be used for ID persistence.

Since the system architecture involves separate Django, FastAPI components and models, testing of each module is important to maintain stability, consistency, and a reliable data processing pipeline. Therefore, Pytest will be adopted for unit testing and coverage reporting across both backend services (not including the model).

2.5.3 Database

- PostgreSQL Database will be used to store all cows' information and behaviour data.

2.5.4 ML Libraries

- PyTorch (training and inference)
- torchvision (pre/postprocessing)
- OpenCV (frame I/O)
- Ultralytics YOLO/Detectron2 or equivalent detector (cow behaviour detection)

2.5.5 Developing Tool

- For both frontend and backend, VS Code will be used as the primary coding environment. While Google Colab will work as the main tool for training the model.

2.5.6 Others

- NVIDIA GPU(s) are required for training and inference

2.6 Project Scheduling

